

Séance 2

Fonctions, gestion des erreurs et récursivité

Notions clés, exemples de code commentés et bonnes pratiques pour structurer, sécuriser et optimiser vos programmes Python.

SOMMAIRE

- 1. Les fonctions
- 2. Gestion des erreurs
- 3. Récursivité et complexité

Partie 1 — Les fonctions

1. Pourquoi découper le code en fonctions ?

Trois bénéfices concrets :

- Réutilisabilité** : on écrit le code une seule fois et on l'appelle autant de fois que nécessaire, sans le recopier.
- Lisibilité** : un programme découpé en fonctions bien nommées se lit presque comme du texte (ex. `calculer_moyenne()`, `afficher_resultat()`), ce qui facilite la compréhension.
- Facilité de débogage et de maintenance** : si un bug apparaît, on sait dans quelle fonction chercher ; et modifier une fonction met à jour tous les endroits où elle est utilisée, sans devoir corriger le code à plusieurs endroits.

2. Paramètre vs valeur de retour

- Un **paramètre** est une donnée qu'on **fournit en entrée** à la fonction, pour qu'elle puisse travailler avec.
- La **valeur de retour** (`return`) est le résultat que la fonction **renvoie en sortie**, une fois son traitement terminé.

En résumé : les paramètres entrent dans la fonction, le `return` en sort.

```
def addition(a, b):    # a et b sont les paramètres
    return a + b      # la valeur renvoyée est le résultat
```

Une fonction **sans `return`** (ou avec un `return` sans valeur) renvoie automatiquement la valeur spéciale **`None`**.

```
def dire_bonjour():
    print("Bonjour")

resultat = dire_bonjour()
print(resultat)    # affiche None
```

3. Qu'est-ce qu'une docstring ?

Une docstring est une chaîne de caractères (souvent entre triples guillemets `"""..."""`) qui **décrit ce que fait une fonction** (son rôle, ses paramètres, ce qu'elle renvoie). Elle se place **juste après la ligne de définition** de la fonction (`def ...:`), avant le reste du code.

```
def carre(x):
    """Renvoie le carré du nombre x."""
    return x * x
```

Elle est utile car elle sert de documentation directement accessible (via `help()` par exemple) et aide les autres développeurs (ou soi-même plus tard) à comprendre la fonction sans lire tout le code.

4. Variable locale vs variable globale

- Une **variable locale** est définie à **l'intérieur d'une fonction**. Elle n'existe que pendant l'exécution de cette fonction et n'est pas accessible depuis l'extérieur.
- Une **variable globale** est définie **en dehors de toute fonction**, au niveau principal du programme. Elle est accessible depuis n'importe quelle partie du code, y compris à l'intérieur des fonctions (en lecture par défaut).

```
x = 10    # variable globale

def exemple():
    y = 5    # variable locale
    print(x, y)    # accès possible à x (global) et y (local)

exemple()
print(y)    # erreur : y n'existe pas en dehors de la fonction
```

Partie 2 — Gestion des erreurs

1. Pourquoi gérer les erreurs avec try/except ?

Gérer les erreurs permet d'**éviter que le programme s'arrête brutalement** lorsqu'une situation imprévue se produit (division par zéro, fichier introuvable, saisie invalide, etc.). Cela permet de :

- garder le contrôle sur le comportement du programme en cas de problème,
- afficher un message clair à l'utilisateur plutôt qu'un plantage,
- continuer l'exécution ou effectuer une action de secours.

Si une **exception n'est pas gérée**, le programme **s'arrête immédiatement** (il « plante ») et Python affiche un message d'erreur (une *traceback*) indiquant le type d'erreur et l'endroit où elle s'est produite.

```
x = 10 / 0    # ZeroDivisionError : le programme s'arrête ici
print("Cette ligne ne sera jamais exécutée")
```

Rôle de `else` et `finally`

- `else`** : le code qu'il contient s'exécute **seulement si aucune exception ne s'est produite** dans le `try`. Utile pour séparer le code « à risque » du code qui doit s'exécuter seulement en cas de succès.
- `finally`** : le code qu'il contient s'exécute **toujours**, qu'il y ait eu une exception ou non. Utile pour les actions de nettoyage (fermer un fichier, une connexion, etc.).

```
try:
    resultat = 10 / 2
except ZeroDivisionError:
    print("Erreur : division par zéro")
else:
    print("Calcul réussi :", resultat)
finally:
    print("Fin du traitement")
```

2. Pourquoi un `except` : nu est-il déconseillé ?

Un `except` : sans préciser le type d'erreur attrape **toutes les exceptions**, y compris celles qu'on n'a pas anticipées et qu'on ne voulait pas forcément masquer (comme des erreurs de programmation, ou même `KeyboardInterrupt` quand l'utilisateur interrompt le programme). Cela peut :

- cacher de vrais bugs** au lieu de les révéler, rendant le débogage très difficile,
- provoquer un comportement inattendu du programme, puisqu'on traite de la même façon des erreurs totalement différentes.

Il est préférable de préciser le(s) type(s) d'erreur attendu(s) :

```
# Déconseillé
try:
    x = 10 / 0
except:
    print("Une erreur est survenue")

# Préférable
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Division par zéro impossible")
```

3. Le module `logging`

Le **logging** est un système permettant d'**enregistrer des messages** sur le déroulement d'un programme (informations, avertissements, erreurs) pendant son exécution, généralement dans un fichier ou dans la console, avec un horodatage et un niveau de gravité.

Pourquoi le préférer à `print` dans un vrai programme :

- on peut **régler le niveau de détail** affiché (ex. cacher les messages de debug en production) sans modifier le code,
- les messages peuvent être **enregistrés dans un fichier** pour être consultés plus tard,
- chaque message peut inclure automatiquement l'heure, le niveau de gravité, le module concerné, etc.,
- `print` mélange tout au même niveau et n'est pas adapté à un usage professionnel ou à grande échelle.

Les niveaux de log (du moins au plus grave) et un usage de chacun :

Niveau	Usage typique
DEBUG	Informations détaillées utiles seulement au développement (ex. valeur d'une variable à un instant donné).
INFO	Informations générales sur le déroulement normal du programme (ex. « Connexion à la base de données réussie »).
WARNING	Signale une situation anormale mais non bloquante (ex. « Fichier de configuration manquant, valeurs par défaut utilisées »).
ERROR	Signale une erreur qui a empêché une opération de se terminer correctement (ex. « Échec de l'enregistrement du fichier »).
CRITICAL	Signale une erreur grave qui peut empêcher le programme de continuer à fonctionner (ex. « Base de données inaccessible, arrêt du programme »).

```
import logging

logging.basicConfig(level=logging.DEBUG)

logging.debug("Variable x = 5")
logging.info("Programme démarré")
logging.warning("Fichier config introuvable, valeurs par défaut utilisées")
logging.error("Échec de l'enregistrement")
logging.critical("Arrêt du programme")
```

Partie 3 — Récursivité et complexité

1. Qu'est-ce que la récursivité ?

La récursivité est une technique où **une fonction s'appelle elle-même** pour résoudre un problème, en le décomposant en sous-problèmes plus petits et similaires au problème d'origine.

Une fonction récursive comporte toujours deux parties :

- Le cas de base (ou condition d'arrêt)** : c'est la condition la plus simple, qui ne nécessite plus d'appel récursif et qui permet de **stopper** la récursion.
- Le cas récursif** : c'est la partie où la fonction **s'appelle elle-même** avec des paramètres qui se rapprochent du cas de base.

```
def factorielle(n):
    if n == 0:        # cas de base
        return 1
    else:             # cas récursif
        return n * factorielle(n - 1)
```

Si le **cas de base est absent** (ou jamais atteint), la fonction s'appelle indéfiniment. Cela provoque une **récursion infinie**, qui finit par épuiser la mémoire d'appel disponible et déclenche une erreur du type `RecursionError: maximum recursion depth exceeded`.

2. Pourquoi utiliser la récursivité ? Est-elle toujours le meilleur choix ?

On utilise la récursivité car elle permet d'écrire un code **plus court, plus clair et plus naturel** pour certains problèmes qui se définissent eux-mêmes de façon répétitive (parcours d'arbres, calculs mathématiques comme la factorielle ou Fibonacci, algorithmes de type « diviser pour régner »).

Elle n'est pas **toujours le meilleur choix**, car :

- elle peut être **plus lente** qu'une version itérative (à cause du coût des appels de fonction),
- elle **consomme plus de mémoire** (chaque appel s'ajoute à la pile d'appels),
- elle peut provoquer une erreur de dépassement de pile si la profondeur est trop grande.

Contre-exemple précis : le calcul de Fibonacci

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Cette version récursive est très inefficace : elle recalcule plusieurs fois les mêmes valeurs (complexité en $O(2^n)$). Une version **itérative** est bien plus rapide et économe en mémoire :

```
def fibonacci_iteratif(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

3. Complexité temporelle et spatiale

- La **complexité temporelle** mesure comment le **temps d'exécution** d'un algorithme évolue en fonction de la taille des données en entrée (n).
- La **complexité spatiale** mesure comment la **quantité de mémoire utilisée** par un algorithme évolue en fonction de la taille des données en entrée (n).

Signification des notations :

Notation	Signification	Exemple
$O(n)$	Le temps (ou la mémoire) croît proportionnellement à n (linéaire).	Parcourir une liste une seule fois.
$O(n^2)$	Le temps croît proportionnellement au carré de n (quadratique).	Deux boucles imbriquées parcourant la même liste.
$O(2^n)$	Le temps double à chaque élément supplémentaire (exponentiel) — devient très vite ingérable.	Fibonacci récursif naïf (sans mémorisation).

4. Mémorisation et `functools.lru_cache`

La **mémorisation** est une technique d'optimisation qui consiste à **stocker la mémoire** les **résultats** déjà calculés d'une fonction, afin de **ne pas les recalculer** si la fonction est appelée à nouveau avec les mêmes arguments. Elle est particulièrement utile pour accélérer les fonctions **récurrentes** comme Fibonacci.

`functools.lru_cache` est un décorateur Python qui **automatise la mémorisation** : il conserve en mémoire les résultats des appels précédents d'une fonction, et renvoie directement le résultat stocké si la fonction est rappelée avec les mêmes arguments, au lieu de refaire le calcul.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Grâce à `lru_cache`, cette version récursive passe d'une complexité en $O(2^n)$ à $O(n)$.

5. Pourquoi Python limite la profondeur de récursion

Python impose une limite (par défaut environ 1000 appels imbriqués) car **chaque appel de fonction récursif consomme de la mémoire** sur la pile d'appels (*call stack*). Sans limite, une récursion infinie ou trop profonde **épuiserait la mémoire** et pourrait faire planter le programme (ou le système) de façon incontrôlée. Cette limite est donc une **sécurité** qui déclenche une erreur propre (`RecursionError`) avant que cela n'arrive.